



Design Patterns

for Scalable Frontend Architecture

React.js as a lens – principles that scale across every framework

إشراف: الدكتور عمر ربحاوي

الهندسة: مُزَن غاوي

فهرس المحاضرة

01 Component-based Architecture

02 لماذا الأنماط التقليدية لم تعد كافية؟

03 Decoupling

04 Separation of Concerns

05 DRY / Reusability

06 Dependency Injection

07 State Ownership

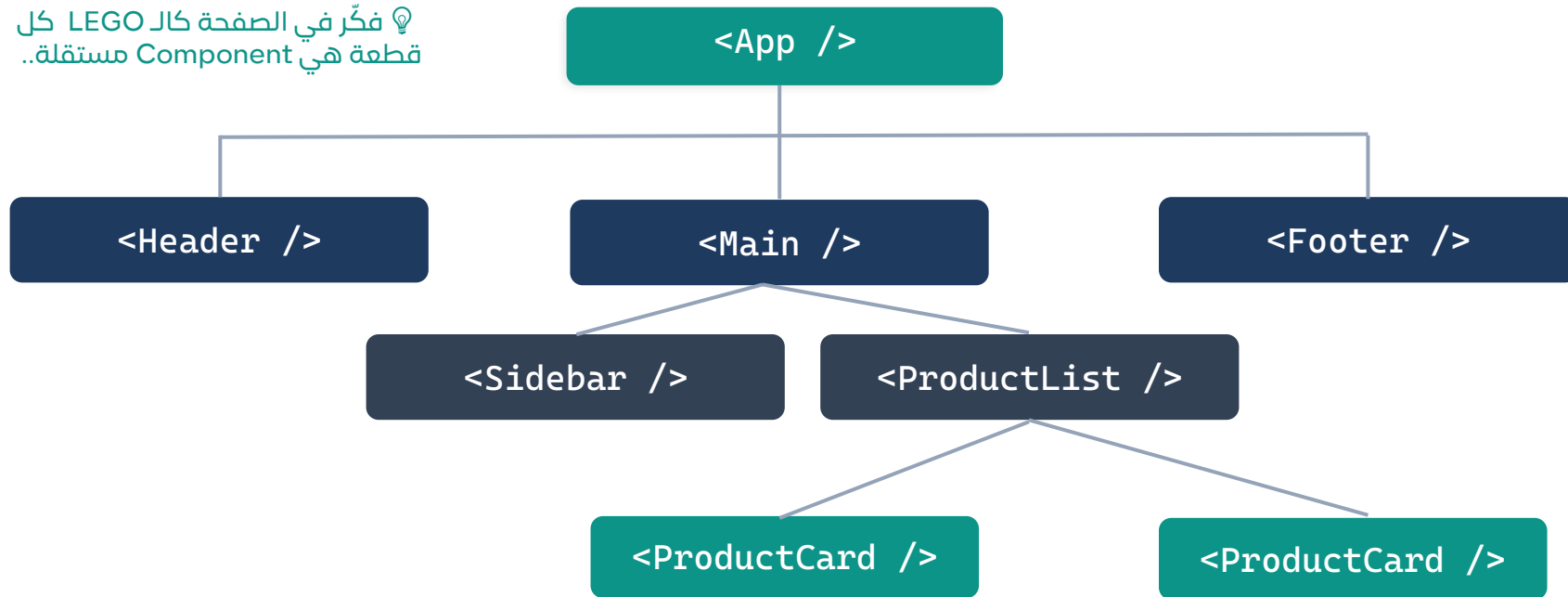
08 Open/Closed

09 Fault Isolation

10 Compute at Source

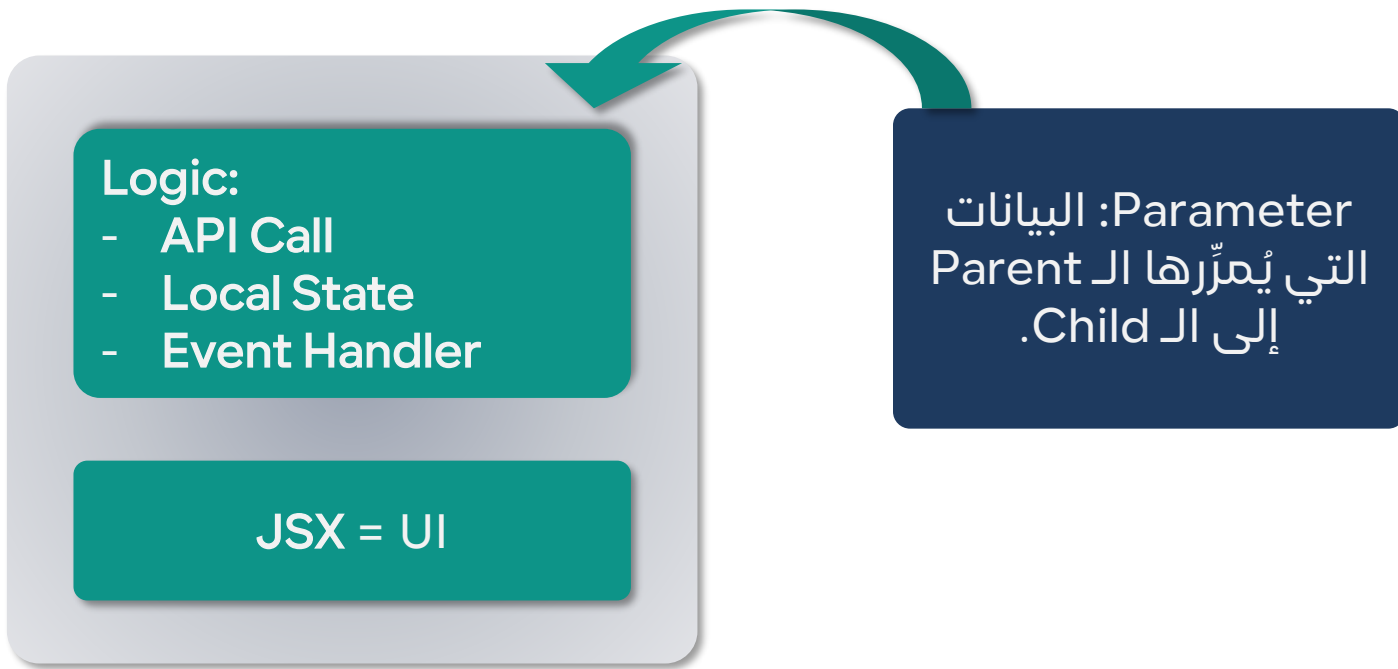
ما هو Component-based Architecture؟

💡 فكّر في الصفحة كالـ LEGO كل قطعة هي Component مستقلة..

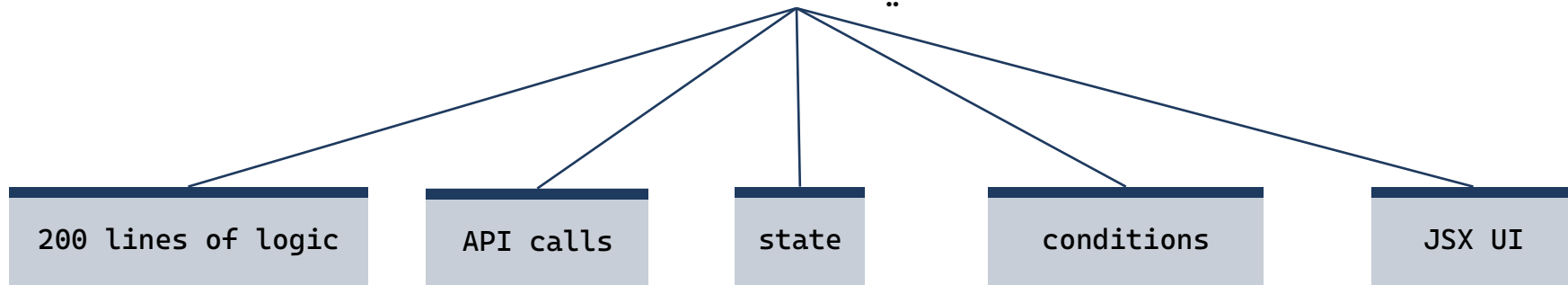


كل صفحة = شجرة من المكونات المستقلة القابلة لإعادة الاستخدام.

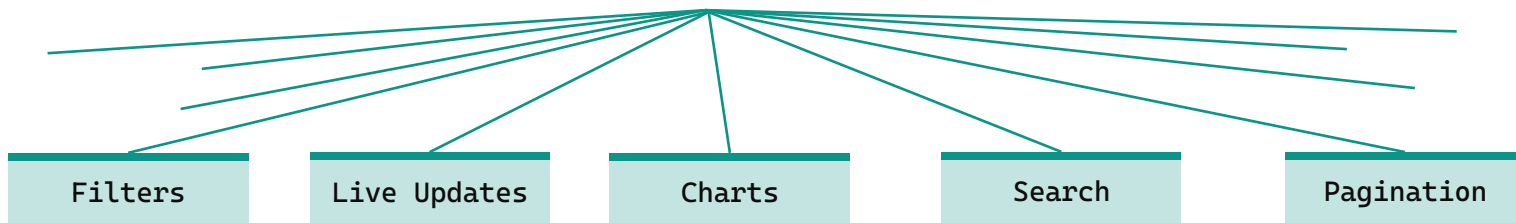
ما هو ال Component؟



عندما نبني Dashboard SaaS



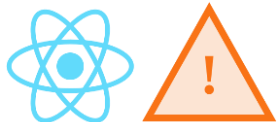
بعد شهرين نضيف ميزات جديدة





لماذا نعاني من مشكلة بطئ الواجهات!



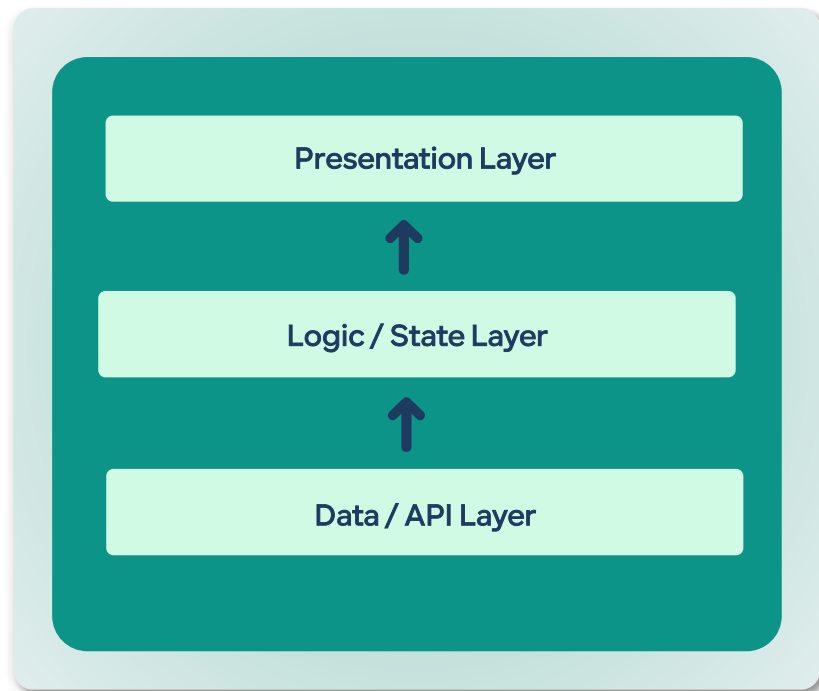


مثال: صفحة عرض المنتجات:





لا أحد يفهم الكود! كل شيء في مكان واحد...





نفس ال Logic مكتوبة في 10 أماكن مختلفة !

صفحة المنتجات

fetch + loading + error — مكتوبة هنا أيضاً

صفحة المستخدمين

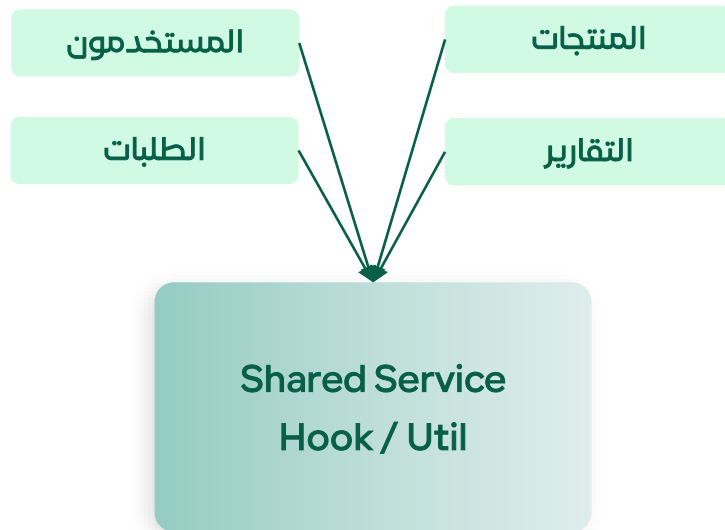
fetch + loading + error — مكتوبة هنا أيضاً

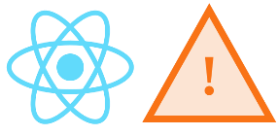
صفحة الطلبات

fetch + loading + error — مكتوبة هنا أيضاً

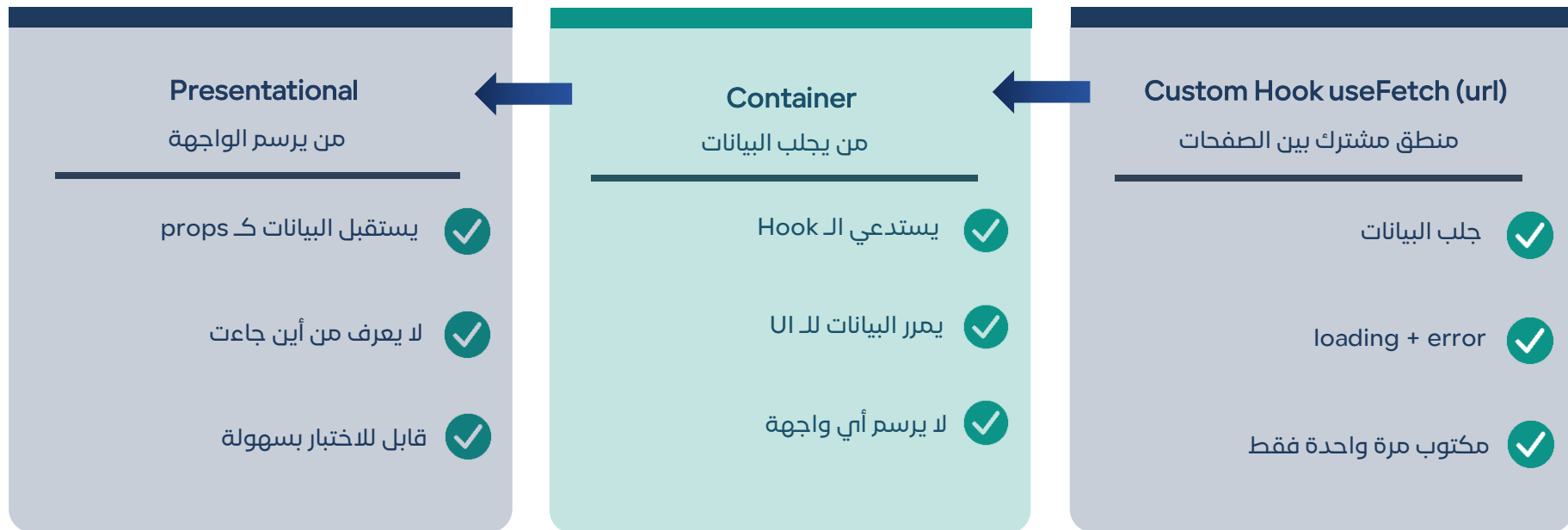
صفحة التقارير

fetch + loading + error — مكتوبة هنا أيضاً



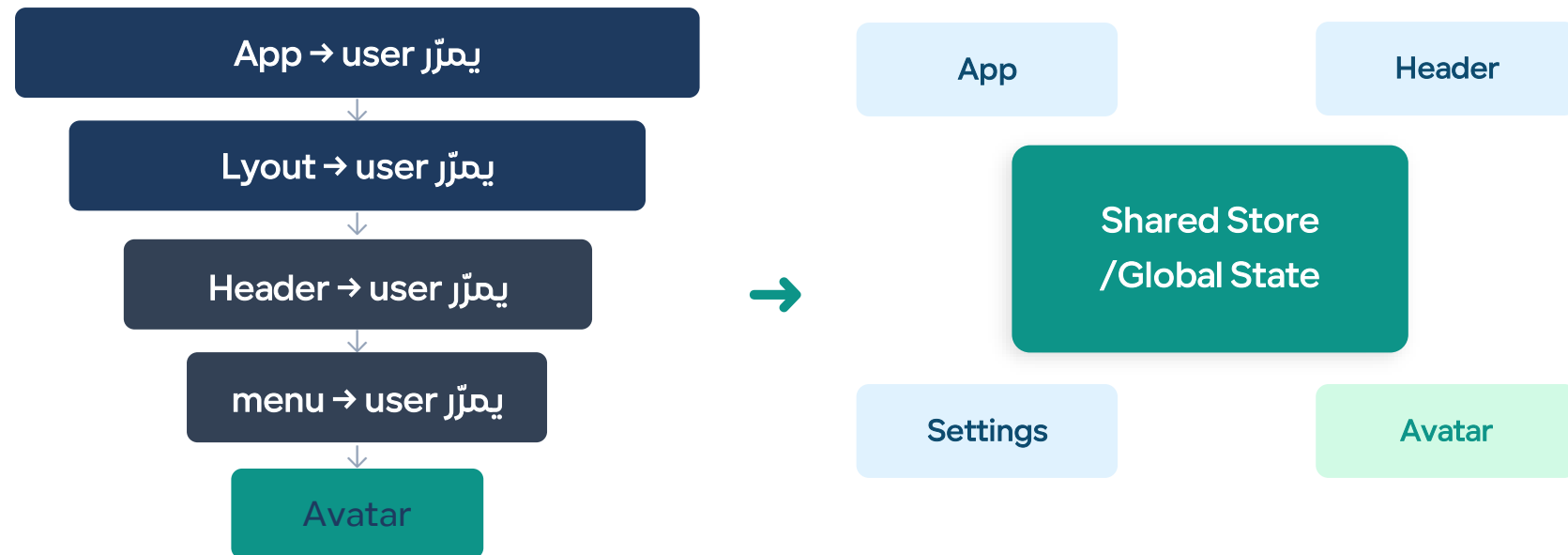


مثال: صفحة المستخدمين:

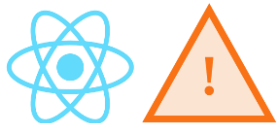




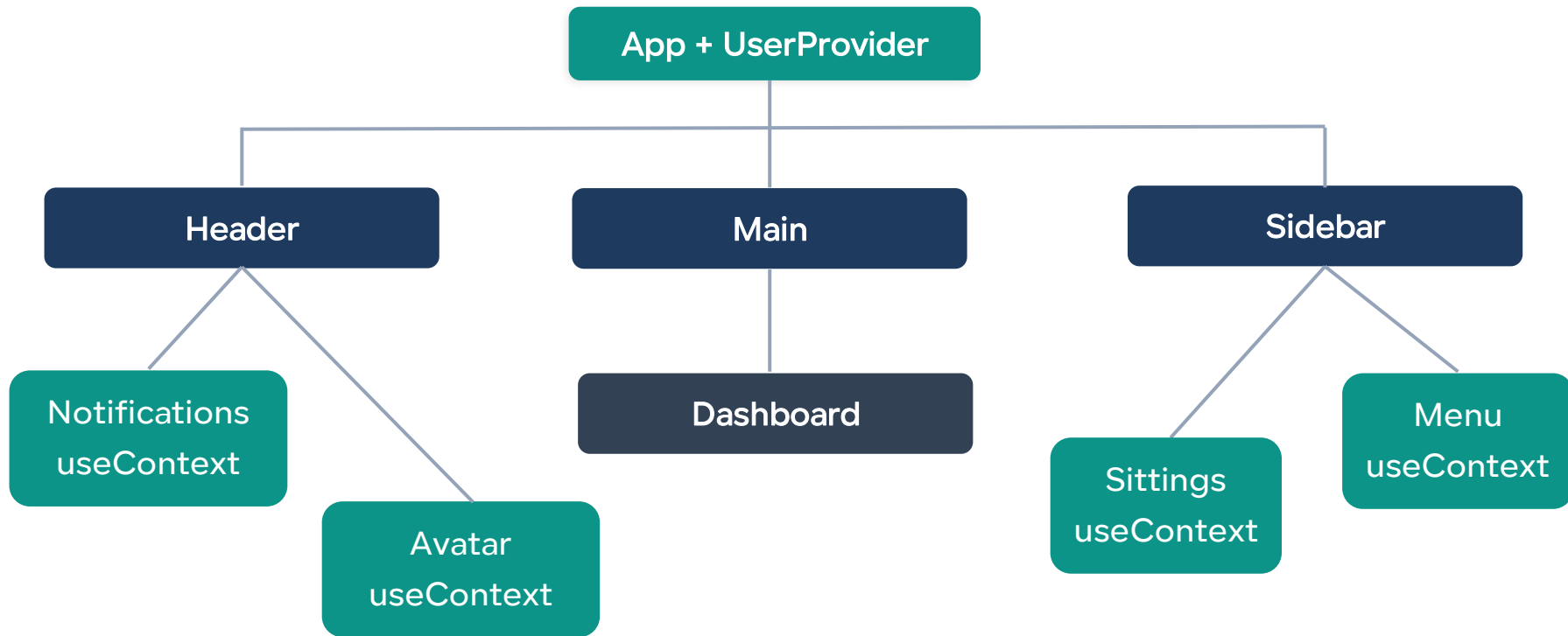
البيانات تعبر عدة طبقات لتصل لمن يحتاجها



كل طبقة تحمل ما لا تستخدمه (Prop Drilling)



مثال: بيانات المستخدم في كل مكان!





من يملك القيمة – النظام أم ال DOM؟



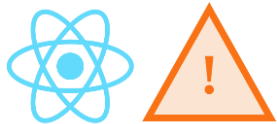
متى كل نهج؟

Uncontrolled

لتقليل عدد عمليات (Re-renders)، وهنا نضحي بجزء من "ملكية الحالة" مقابل الأداء

Controlled

Validation • Autocomplete • تفعيل/تعطيل زر



مثال: صفحة تسجيل دخول:

Uncontrolled

Email:

user@...

كلمة المرور:

.....

تسجيل الدخول

بعد الضغط:
"البريد غير صحيح"
(الخطأ جاء متأخراً)



Controlled

Email:

user@example.com ✓

كلمة المرور:

أحرف مطلوبة 5+ ← ...

تسجيل الدخول (معطل حتى الصحة)

أثناء الكتابة:
لحظي Validation
زر يُفعّل عند اكتمال البيانات



مكوّنات صلبة, كل ميزة جديدة تكسر ما قبلها!

Card Component

- `showBadge={true}`
- `badgeColor='blue'`
- `showPrice={false}`
- `onBuyClick={...}`
- `actionsLayout='row'`
- `showFavorite={true}`
- `+20 prop آخر...`



`Card.Badge` → اختياري, قابل للتخصيص

`Card.Image` → اختياري, قابل للتخصيص

`Card.Title` → اختياري, قابل للتخصيص

`Card.Price` → اختياري, قابل للتخصيص

`Card.Actions` → اختياري, قابل للتخصيص



خطأ في مكوّن واحد يُسقط الصفحة كاملة

صفحة Dashboard

الـ Header آمن، لا يحتاج boundary

⬇️ الحاجز يُغلّف كل قسم خطير

ErrorBoundary

Charts

فشلت API ← أرجع خطأ

Fallback: ← رسالة ودية

ErrorBoundary

جدول المستخدمين

يعمل طبيعياً

لم يتأثر بسقوط Charts

لم يتأثر — يعمل طبيعياً Footer



Bundle ثقيل, SEO ضعيف...

المتصفح

تحميل JS Bundle الضخم, وقت طويل...

يبدأ التشغيل, وقت إضافي...

يرسل طلب لل API, شبكة!

يعرض البيانات



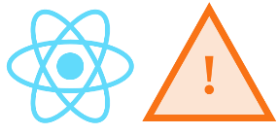
الخادم

يجلب البيانات مباشرة

يبنى HTML جاهزاً على الخادم

يرسل HTML نهائياً للمتصفح

المستخدم يرى المحتوى فوراً SEO



مثال: قائمة المنتجات مع Infinite Scroll + SEO

صفحة المنتجات - ProductsPage

Server Component

تحميل المنتجات بشكل أسرع, SEO ممتاز

HeroSection, عنوان + وصف

Initial Products Grid, أول 20 منتج من DB مباشرة

أقسام ثابتة

SEO Meta

Client Component

تفاعل المستخدم فقط

SearchBar
بحث فوري

FilterPanel
فلتر ديناميكية

InfiniteScroll
تحميل المزيد

الخلاصة

مهما اختلفت الأدوات

تبقى المشاكل ثابتة والحلول موحدة 🧐

